PUNTO 1 – CONSULTA TEÓRICA

Listas enlazadas simple, doble y circular

Hector Ortega – Jhon Del Castillo

Thomas obando

Taller de Estructuras de Datos

1. ¿Qué es un nodo y qué atributos tiene en cada tipo de lista?

Un nodo es la unidad básica de una lista enlazada. Contiene un dato y una o más referencias que permiten conectarlo con otros nodos. En una lista simple, cada nodo tiene el dato y una referencia al siguiente nodo. En una lista doble, además de la referencia al siguiente, tiene una referencia al nodo anterior. En una lista circular, el último nodo no apunta a None, sino que vuelve a conectar con el primer nodo.

Tipo	Dato	Siguiente	Anterior
Simple	Sí	Sí	No
Doble	Sí	Sí	Sí
Circular	Sí	Sí	No

2. ¿En qué se diferencia una lista enlazada de un arreglo en memoria?

La diferencia principal está en la forma de organizar y acceder a los elementos. Un arreglo almacena sus elementos en posiciones contiguas de memoria, por lo que permite acceder rápidamente a un elemento mediante su índice. Una lista enlazada almacena los nodos de forma dinámica y cada nodo utiliza referencias para indicar dónde está el siguiente elemento.

Ventajas de las listas enlazadas: pueden crecer o disminuir dinámicamente y las inserciones o eliminaciones no requieren desplazar todos los elementos posteriores. Desventajas: para buscar un elemento por posición normalmente hay que recorrer los nodos desde el inicio y cada nodo necesita memoria adicional para sus referencias.

Ventajas de los arreglos: acceso directo por índice y buena localidad de memoria. Desventajas: su tamaño puede ser fijo según la implementación y las inserciones o eliminaciones en posiciones intermedias pueden requerir desplazar elementos.

3. ¿Qué cambia en una lista doble respecto a una simple?

La lista doble agrega al nodo una segunda referencia llamada anterior. Así, cada nodo puede conectarse tanto con el nodo siguiente como con el nodo anterior. Esto permite recorrer la estructura en ambos sentidos y facilita ciertas eliminaciones cuando ya se tiene una referencia al nodo que se desea eliminar.

El costo es un campo de referencia adicional por nodo. Por lo tanto, una lista doble utiliza más memoria que una lista simple. También hay que actualizar más enlaces al insertar o eliminar, porque deben mantenerse correctamente las referencias siguiente y anterior.

4. ¿Cómo se identifica el final de una lista circular y cómo se evita un ciclo infinito?

En una lista circular no se utiliza None para indicar el final. El último nodo apunta nuevamente al primero. Por esta razón, un recorrido no debe utilizar únicamente una condición como “mientras el nodo no sea None”, porque nunca llegaría a None.

Una forma de evitar el ciclo infinito es guardar una referencia al nodo inicial y detener el recorrido cuando se vuelva a encontrar ese mismo nodo. Otra opción es conocer el tamaño de la lista y realizar exactamente esa cantidad de pasos.

5. ¿Qué complejidad Big O tienen las operaciones?

Las complejidades dependen de si la estructura conserva únicamente la cabeza o también una referencia a la cola. Para insertar al inicio se puede enlazar el nuevo nodo directamente con la cabeza, por lo que el costo es O(1). Insertar al final es O(n) si hay que recorrer desde la cabeza hasta el último nodo; si se mantiene una referencia a la cola, puede ser O(1).

Operación	Lista simple	Lista doble	Lista circular
Insertar al inicio	O(1)	O(1)	O(1) con cola
Insertar al final	O(n) / O(1) con cola	O(n) / O(1) con cola	O(1) con cola
Buscar	O(n)	O(n)	O(n)
Eliminar por dato	O(n)	O(n)	O(n)

En las listas doble y circular, mantener una referencia a la cola permite llegar directamente al último nodo. Sin embargo, buscar un dato sigue siendo O(n) porque, en el peor caso, es necesario revisar los nodos uno por uno.

6. Dos aplicaciones reales de cada tipo de lista

Lista enlazada simple: 1. Manejo de secuencias donde solo interesa avanzar en una dirección. 2. Implementación de estructuras como pilas o colas mediante nodos enlazados.

Lista enlazada doble: 1. Historial de navegación, donde se puede avanzar y retroceder. 2. Sistemas de reproducción o navegación entre elementos que requieren ir al anterior y al siguiente.

Lista enlazada circular: 1. Planificación de turnos o procesos de forma repetitiva. 2. Implementación de recorridos cíclicos, como listas de reproducción o el problema de Josefo.

7. Diagramas de nodos

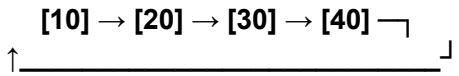
Lista simple:

[10] → [20] → [30] → [40] → None

Lista doble:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

Lista circular:



8. Tabla comparativa

Característica	Simple	Doble	Circular	Memoria	Uso
Nodo	Dato + siguiente	Dato + siguiente + anterior	Dato + siguiente	Menor	Recorrido en una dirección
Recorrido	Hacia adelante	Adelante y atrás	Cíclico	Simple: menor	Turnos y secuencias
Insertar inicio	O(1)	O(1)	O(1) con cola	Depende de referencias	Frecuente en estructuras dinámicas
Buscar	O(n)	O(n)	O(n)	Mayor en doble	Cuando el acceso por índice no es prioridad
Eliminar	O(n) por dato	O(n) por dato	O(n) por dato	Mayor en doble	Manipulación dinámica

9. Referencias

Python Software Foundation. (s. f.). Data Structures. Python Documentation. https://docs.python.org/3/tutorial/datastructures.html

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Nota: Este documento desarrolla el Punto 1 solicitado en el taller: preguntas teóricas, diagramas, tabla comparativa y referencias. Se mantiene la terminología de lista simple, doble y circular utilizada en el documento del taller.